

White Wall Studios — System Design Document

Last updated: 2026-03-21 **Domain:** whitewallstudios.co **Repository:** white-wall-mockup (GitHub → Vercel) **Authors:** Andrew (developer), Drew (business owner), Max (technical stakeholder)

Table of Contents

1. System Overview
 2. Infrastructure
 3. Booking Flow
 4. Payment Architecture
 5. Acuity Scheduling Integration
 6. QuickBooks Integration
 7. Email Notifications
 8. Client-Side Architecture
 9. Safety & Risk Analysis
 10. Costs
 11. Environment & Deployment
 12. Future Roadmap
 13. Critical Warnings & Operational Rules
-

1. System Overview

What It Is

A custom-built booking website for **White Wall Studios**, a self-service photo/video studio with two locations in the Greenville, SC area. The site replaces a previous Squarespace site and provides a fully custom booking experience with integrated payment processing.

Who It Serves

- **Customers** — photographers, videographers, event planners booking studio time
- **Drew (owner)** — manages bookings via Acuity dashboard, receives payments via Square
- **Max (technical stakeholder)** — infrastructure and integration oversight

Architecture Overview

CUSTOMER BROWSER

```
index.html  powdersville.html  taylor-mill.html
gallery.html  book-powdersville.html  book-taylor-mill.html
booking-config.js  booking-flow.js
```

HTTPS

VERCEL (Hobby Plan)

Static HTML/CSS/JS

12 HTML
pages
+ styles/
+ scripts/
+ images/
+ fonts/

Serverless Functions (api/)

availability-dates.js
availability-times.js
verify-availability.js
create-checkout.js
booking-callback.js
notify-owner.js
qbo-auth.js / qbo-callback.js

Acuity Scheduling

- Availability
- Appointments
- Add-ons
- Calendars
- Email confirm

Square Payments

- Payment Links
- Order verify
- Refunds

Resend (email API)

Owner alerts
for 25+ ppl

QuickBooks

(auto-sync)

Tech Stack

Layer	Technology
Frontend	Vanilla HTML/CSS/JS (no framework)
Hosting	Vercel (Hobby plan)
Serverless	Vercel Functions (Node.js 18+)
Scheduling	Acuity Scheduling API (v1, REST)
Payments	Square Checkout API (Payment Links)
Email alerts	Resend API
Invoicing	QuickBooks Online (auto-sync from Acuity)
DNS	GoDaddy
Fonts	Self-hosted in /fonts/
Dependencies	puppeteer (for PDF generation only, not runtime)

2. Infrastructure

Hosting — Vercel

- **Plan:** Hobby (free tier)
- **Project name:** white-wall-mockup
- **Production URL:** <https://white-wall-mockup.vercel.app>
- **Custom domain:** whitewallstudios.co
- **Deployment:** Auto-deploy on push to main branch
- **Serverless region:** Default (auto)
- **Clean URLs:** Enabled (`vercel.json` → `cleanUrls: true`)
- **Trailing slashes:** Disabled

DNS — GoDaddy

Domain whitewallstudios.co managed through GoDaddy, with DNS records pointing to Vercel.

Redirects (`vercel.json`)

Source	Destination	Status
<code>/contact</code>	<code>/#contact</code>	301
<code>/white</code>	<code>/gallery</code>	301
<code>/booking-studio</code>	<code>/book-powdersville</code>	301
<code>/booking-powdersville</code>	<code>/book-powdersville</code>	301
<code>/booking-taylors-mill</code>	<code>/book-taylors-mill</code>	301
<code>/gear-rentals</code>	<code>/gear-rentals-powdersville</code>	301
<code>/beauty-area</code>	<code>/powdersville</code>	301
<code>/membership-subscriptions</code>	<code>/</code>	301
<code>/pricing</code>	<code>/</code>	301

Cache Headers (`vercel.json`)

Path	Cache-Control
<code>/images/*</code>	<code>public, max-age=31536000, immutable</code>
<code>/fonts/*</code>	<code>public, max-age=31536000, immutable</code>
<code>/scripts/*</code>	<code>public, max-age=0, must-revalidate</code>

Environment Variables (Vercel)

Variable	Source	Secret?	Notes
<code>ACUITY_USER_ID</code>	Acuity > Integrations > API	No	Value: 36967128
<code>ACUITY_API_KEY</code>	Acuity > Integrations > API	Yes	HTTP Basic auth password
<code>SQUARE_ACCESS_TOKEN</code>	Square Developer Dashboard	Yes	Currently sandbox token

Variable	Source	Secret?	Notes
SQUARE_LOCATION_ID	Square Dashboard > Locations	No	Sandbox: LTPQKY2V3NOAH
SQUARE_ENVIRONMENT	Manual	No	sandbox or production
BOOKING_SECRET	openssl rand -hex 32	Yes	HMAC signing key for state
RESEND_API_KEY	Resend Dashboard	Yes	Email sending API key
NOTIFICATION_EMAIL	Manual	No	Drew's email for alerts
QBO_CLIENT_ID	Intuit Developer Portal	No	QuickBooks OAuth2 app
QBO_CLIENT_SECRET	Intuit Developer Portal	Yes	QuickBooks OAuth2 secret

Third-Party Services

Service	Role	Auth Method
Acuity Scheduling	Calendar, availability, appointments	HTTP Basic (userId:apiKey)
Square	Payment processing via Payment Links	Bearer token
Resend	Transactional email (owner alerts)	Bearer token
QuickBooks Online	Invoice auto-sync from Acuity	OAuth2 (pending production)
GoDaddy	Domain registration and DNS	Dashboard
Vercel	Hosting, serverless functions, CDN	Git integration

3. Booking Flow (The Critical Path)

Architecture: Pay then Book

The core design principle: **no appointment exists until the customer has paid**. This prevents premature emails, phantom bookings, and unpaid slot-holds.

Customer Browser	Vercel Serverless	Square	Acuity
1. Pick duration			
2. Pick date/time	GET /availability (proxy to Acuity)		
3. Fill details			
4. Sign waiver			
5. Pick add-ons			

```

6. Click Pay & Book      POST /verify-avail

                           POST /create-checkout
                           (HMAC sign state)      Create Payment
                                           Link
                           Return checkout URL

REDIRECT TO SQUARE

(customer pays on Square's hosted page)

SQUARE REDIRECTS BACK
GET /api/booking-callback?orderId=X&state=Y&sig=Z

7. Verify HMAC sig
8. Verify order          GET /orders/{id}
                           (COMPLETED)
9. Create appt

10. Notify owner
302 → /confirmation

Sends
confirm
email

```

Step-by-Step User Journey

Step 1 — Timing (client-side) The customer selects a session duration. Each duration maps to an Acuity appointment type ID. Selecting a duration auto-advances to Step 2.

Step 2 — Schedule (client-side + API)

- Calendar renders for the current month
- Client calls `GET /api/availability-dates?appointmentTypeID={id}&month={YYYY-MM}`
- Serverless function proxies to Acuity `GET /availability/dates`
- Available dates highlighted; customer clicks a date
- Client calls `GET /api/availability-times?appointmentTypeID={id}&date={YYYY-MM-DD}`
- Time slots rendered as pills; customer selects one

Step 3 — Details (client-side) Contact info form: first name, last name, email, phone, notes. Intake form: business name, total participants, Instagram handle (required), email acknowledgment signature. Terms & conditions acceptance via typed signature matching full name.

Step 4 — Waiver (client-side) Full liability waiver text displayed. Customer clicks “Sign as [Name]” button. Electronic signature has same legal force as handwritten (per waiver text).

Step 5 — Add-ons & Pay (client-side + API) Add-on selection (backdrops, lighting, walls, chairs, tables, TV, PA). Order summary displays with session + add-on subtotals.

When customer clicks “Pay & Book”:

1. **Client-side validation** — checks all steps complete
2. `POST /api/verify-availability` — confirms slot still open
3. `POST /api/create-checkout` — server builds Square line items and creates payment link
4. **Browser redirects to Square’s hosted checkout page**
5. Customer enters card, pays
6. **Square redirects to `GET /api/booking-callback`** with `orderId`, `transactionId`, `checkoutId`, `referenceId`
7. Callback verifies HMAC signature on `state` param
8. Callback verifies Square order is `COMPLETED` or `OPEN`
9. Callback creates Acuity appointment with all data
10. Callback fires owner notification (async, non-blocking)
11. Redirect to `/booking-confirmation`

HMAC Signing for Tamper Protection

The booking state (contact info, add-ons, datetime, etc.) must survive the roundtrip through Square’s redirect. It’s encoded and signed:

`create-checkout.js`:

```
payload = JSON.stringify(bookingState)
sig = HMAC-SHA256(BOOKING_SECRET, payload)
encoded = base64url(payload)
redirectUrl = /api/booking-callback?state={encoded}&sig={sig}
```

`booking-callback.js`:

```
payload = base64url.decode(state)
expected = HMAC-SHA256(BOOKING_SECRET, payload)
timingSafeEqual(sig, expected) // constant-time comparison
bookingState = JSON.parse(payload)
```

This prevents a malicious user from modifying the booking state (e.g., changing the price or appointment type) between checkout creation and callback.

Data Flow Summary

Data	Where it lives	Who sets it
Session duration + price	<code>booking-config.js</code> (client display), <code>acuity.js</code> <code>SESSION_PRICES</code> (server truth)	Developer
Add-on selections	Client state → HMAC-signed through Square redirect	Customer
Contact info	Client state → HMAC-signed → Acuity appointment	Customer
Payment amount	Server-side <code>buildSquareLineItems()</code> — NOT from client	Server

Data	Where it lives	Who sets it
Appointment	Acuity (created only after payment verified)	Server
Confirmation email	Acuity (auto-sent on appointment creation)	Acuity
Invoice	QuickBooks (auto-created by Acuity sync)	Acuity

4. Payment Architecture

Square Integration

Detail	Value
API version	2026-01-22 (via <code>Square-Version</code> header)
Sandbox base URL	<code>https://connect.squareupsandbox.com</code>
Production base URL	<code>https://connect.squareup.com</code>
Sandbox location ID	LTPQKY2V3NOAH
Processing fee	2.9% + \$0.30 per transaction
SDK	None — raw <code>fetch()</code> to avoid BigInt serialization issues on Vercel

How Payment Links Work

1. `create-checkout.js` calls `buildSquareLineItems()` with the appointment type ID and add-ons
2. Each line item has a `name`, `amount` (cents), and `quantity`
3. `createPaymentLink()` sends `POST /v2/online-checkout/payment-links` to Square
4. Square returns a `checkoutUrl` — a hosted payment page
5. Customer pays on Square's page (we never see card numbers)
6. Square redirects back to our callback URL with query params

How Line Items Are Built (Server-Side)

The `buildSquareLineItems()` function in `acuity.js` is the **authoritative pricing source**. It reads from `SESSION_PRICES` and `ADDON_PRICES` constants — not from the client request. The client sends add-on selections (which backdrops, how many tables), but the server looks up the correct price.

Example line items for a 2hr PV session + lighting + 2 backdrops:

```
[
  { "name": "2 Hour Session", "amount": 20000, "quantity": 1 },
  { "name": "Lighting Rental", "amount": 10000, "quantity": 1 },
  { "name": "Single Backdrop", "amount": 1500, "quantity": 2 }
]
```

Square Endpoints Used

Endpoint	Method	Purpose
/v2/online-checkout/payment-link	POST	Create a payment link
/v2/orders/{id}	GET	Verify order is COMPLETED after payment
/v2/online-checkout/payment-link/{id}	DELETE	Invalidate a payment link (available but not used in production)
/v2/refunds	POST	Refund a payment (available but manual process)

Refund Process

Refunds are **manual via Square Dashboard**. The only scenario requiring a refund is if the Acuity appointment creation fails after payment (extremely rare slot conflict). Drew handles this directly.

The `refundPayment()` function exists in `square.js` for future automation but is not called in any production flow.

5. Acuity Scheduling Integration

API Details

Detail	Value
Base URL	<code>https://acuityscheduling.com/api/v1</code>
Auth	HTTP Basic over SSL (<code>userId:apiKey</code>)
Owner ID	24638772 (used in scheduler URLs)
User ID	36967128 (used for API auth)
Plan	Business (via Squarespace), <code>can_use_api: true</code>
Timezone	<code>America/New_York</code> (both locations)

Calendar IDs

Location	Calendar ID
Powdersville	6255578
Taylor's Mill	6252295

Appointment Type IDs

Powdersville (Calendar 6255578)

Duration	Type ID	Price
1 hour	89113040	\$130
2 hours	89113116	\$200
3 hours	89114444	\$270
4 hours	89114517	\$350
6 hours	89114539	\$500
Full day (18hr)	89114581	\$980

Taylor's Mill (Calendar 6252295)

Duration	Type ID	Price
1 hour	38342199	\$110
2 hours	28312352	\$170
3 hours	28312534	\$230
4 hours	28312549	\$280
6 hours	36030598	\$420
Full day (12hr)	28312569	\$550

Add-On ID Mappings

Key	Acuity Add-On ID	Name	Price
lighting-powdersville	6723268	Lighting Package (2 Fixtures)	\$100 (Acuity) / \$125 (site)
lighting-taylors-mill	12387016	Lighting Rental	\$50
backdrops-all	6840261	All Backdrops	\$50
backdrops-single	6840263	Single Backdrop	\$15
walls-all	6840264	All Rolling Walls	\$70
walls-single	6840265	Single Rolling Wall	\$30
chairs-25	6840270	25 Chairs	\$100
chairs-50	6840271	50 Chairs	\$190
chairs-75	6840272	75 Chairs	\$280
chairs-100	6840274	100 Chairs	\$370
table	6840275	8ft Folding Table	\$15
tv	6840276	86in Rolling TV	\$50
pa-system	6840278	PA System	\$40

Legacy add-on (to be deleted): ID 2592725 — “Paper Backdrop” (\$20)

Pricing discrepancy: Powdersville lighting is \$100 in Acuity but \$125 on the site. Drew needs to update Acuity to \$125.

How Add-Ons Are Passed to Acuity

Acuity's add-on system is flat — no quantities, no variants. We work around this:

- **Quantity items** (backdrops, walls, tables): Pass the same add-on ID multiple times in the `addonIDs` array. Example: 3 single backdrops = [6840263, 6840263, 6840263]. Acuity charges \$15 x 3 = \$45.
- **“All” bundles** (all backdrops, all walls): Single add-on ID at bundle price.
- **Tiered items** (chairs): Separate add-on per tier (25/50/75/100).
- **Toggle items** (lighting, TV, PA): Single add-on ID, present or absent.

Specific selections (which backdrop colors, which wall numbers) go into appointment **notes** since Acuity add-ons are just price line items with no structured metadata.

Intake Form Field IDs

Field	Acuity ID	Form
Business Legal Name	10764621	1935872 (Intake Form)
Total Number of Participants	10764623	1935872
Instagram Handle	10764624	1935872
Will you read the entire email...	10947712	1935872
PV: I have read and agree (T&C)	18026152	3189363 (PV Terms)
TM: I have read and agree (T&C)	10764522	1935852 (TM Terms)
TM: I will only walk to WhiteWall...	18026602	1935852

Fields are passed as `{id, value}` objects (not by label).

Notes Construction

`buildAppointmentNotes()` produces a structured text block:

Event booking: Yes
 Event guests: 75
 Event description: Birthday party

Add-ons:
 Backdrop colors: Black, Pink
 Lighting rental: Yes
 Rolling walls: Wall 1, Wall 3
 Chairs: 50
 Tables: 3
 86in TV: Yes

Booked via `whitewallstudios.co`

Capacity/traffic alerts appended by `booking-callback.js`:

[CAPACITY ALERT: 75 participants - follow-up required]
 Customer note: Large birthday party, need extra setup time

The `?admin=true` query param on POST `/appointments` is **required** for the `notes` field to be saved.

Undocumented Behaviors We Rely On

1. **noPayment: true** — Creates appointment without requiring payment. Not in official docs.
2. **?admin=true query param** — Required for **notes** field to persist. Without it, notes are silently dropped.
3. **Duplicate addonIDs for quantity** — Same ID x N charges N x price. Response deduplicates IDs but price math is correct.
4. **fields accepts {id, value}** — Docs mention **label** but ID-based works and is more reliable.

Things That Do NOT Work (Tested and Confirmed)

What	Expected	Actual	Date Tested
noEmail param on POST /appointments	Suppress confirmation email	Emails still sent	2026-03-17
POST /appointments/{id}/payments	Record external payment	500 Internal Server Error for ALL source values (tested 50+)	2026-03-19, re-confirmed 2026-03-20
PUT /appointments/{id} with paid or price	Mark as paid / change price	Silently ignored (read-only fields)	2026-03-19
Scheduler URL add-on params (addonIDs[], addon[], addons[])	Pre-select add-ons	All ignored. Private add-ons hidden on public scheduler.	2026-03-20
confirmationPagePaymentCancel link when invalidation	Cancel link when appointment cancelled	Payment page still accepts card input after cancellation	2026-03-17

6. QuickBooks Integration

Current State

Acuity auto-syncs appointments to QuickBooks. The setting was changed to “**Create a draft invoice**” (2026-03-20) so no invoice email is sent to the customer.

- **Draft invoices** appear in QuickBooks as unpaid
- Drew confirmed he **does not want customers invoiced** — prefers they don’t get invoices
- The draft is a byproduct of the Acuity-QuickBooks sync, not an intentional customer touch-point

Integration Map

Connection	Direction	What it does
Acuity → QuickBooks	Auto-sync	Creates draft invoices on appointment creation; auto-voids on cancellation
Square → QuickBooks	Auto-sync	Syncs payment transactions
Our App → QuickBooks	Pending	Would mark draft invoices as paid (requires production API credentials)

OAuth2 Endpoints (Implemented, Not Yet in Production)

Endpoint	Purpose
GET /api/qbo-auth	Redirects Drew to Intuit's OAuth2 consent page
GET /api/qbo-callback	Exchanges auth code for tokens, displays them for manual storage

What's Needed for Production

1. Complete the **Intuit Developer compliance questionnaire** (~30 min)
2. Get production API credentials
3. Drew authorizes via /api/qbo-auth
4. Store tokens in Vercel env vars (QBO_ACCESS_TOKEN, QBO_REFRESH_TOKEN, QBO_REALM_ID)
5. Booking callback queries QuickBooks for draft invoice by customer + date, then records payment

Tested successfully in sandbox (2026-03-20).

7. Email Notifications

Resend Integration (Owner Alerts)

File: `api/notify-owner.js`

Sends email to Drew for high-traffic bookings. Called from `booking-callback.js` as a **fire-and-forget** async call — notification failure never blocks the booking flow.

Env Var	Purpose
RESEND_API_KEY	Resend API authentication
NOTIFICATION_EMAIL	Drew's email address

From address: WhiteWall Studios <contact@whitewallstudios.co>

When Emails Are Sent

Condition	Subject Line	Content
25-49 participants	[White Wall] High Traffic Booking - {N} participants	Customer details, datetime, location, customer note. "A cleaning fee may apply."
50+ participants	[White Wall] CAPACITY ALERT - {N} participants	Same as above, plus "This booking has 50+ participants and requires follow-up."
< 25 participants	No email sent	—

Email Body Format (Plain Text)

CAPACITY ALERT - FOLLOW-UP REQUIRED

Customer: Jane Smith
Email: jane@example.com
Phone: 864-555-1234
Location: Powdersville
Date/Time: 2026-04-15T14:00:00-0400
Participants: 75
Appointment ID: 123456789

Customer's note:
Large birthday party, need extra setup time

This booking has 50+ participants and requires follow-up.

Acuity Confirmation Emails

Acuity **automatically sends a confirmation email** when an appointment is created via the API. This is by design — the email goes out at exactly the right time (after payment) because we only create the appointment after Square confirms payment.

There is **no way to suppress this email** via the API. The `noEmail` param does not work (tested 2026-03-17).

8. Client-Side Architecture

File Structure

```
scripts/  
  booking-config.js - Location data, pricing, add-on definitions, Acuity mappings  
  booking-flow.js   - 5-step booking UI state machine and rendering  
styles/  
  booking.css       - Calendar, time slots, spinner, add-on card styles
```

booking-config.js — What It Controls

Global config object `window.WWS_BOOKING_CONFIG` with:

- **Locations array** (Powdersville first, per Drew’s directive)
 - slug, name, accent color, address, eyebrow text
 - description and policies arrays
 - durations — each with id, label, hours, price, description, supportsEvents, acuityTypeKey
 - addons — each with id, type, name, image, pricing, and type-specific config
- **Integrations config**
 - `acuity.enabled`, `acuity.mode` (“api”), `acuity.locations` mapping duration IDs to appointment type IDs
 - `square.enabled`, `square.mode` (“payment-links”)

booking-flow.js — State Machine

The booking flow is a **5-step state machine** implemented as an IIFE:

Step	Name	What happens
1	Timing	Duration selection. Auto-advances to Step 2 on click.
2	Schedule	Calendar + time slot picker. Fetches availability from API.
3	Details	Contact info, intake form, T&C signature, email acknowledgment.
4	Waiver	Liability waiver display + electronic signature.
5	Add-ons	Add-on carousel/selection + order summary + “Pay & Book” button.

State Object

```
{
  step: 1,
  durationId: "",
  eventIntent: "no",
  participants: "",
  eventDescription: "",
  highTrafficNote: "",
  acknowledgements: { cleanup: false, capacity: false, selfService: false },
  contact: { firstName: "", lastName: "", email: "", phone: "", notes: "" },
  intake: { business: "", participants: "", instagram: "", readEmail: false },
  emailAcknowledgment: "",
  termsSignature: "",
}
```

```

waiverSigned: false,
addons: {},          // initialized from location.addons config
selectedDate: "",
selectedTime: "",
availableDates: [],
availableTimes: [],
calendarMonth: "YYYY-MM",
isLoadingDates: false,
isLoadingTimes: false,
isSubmitting: false
}

```

Step Validation Logic Steps are gated — you cannot skip ahead past an incomplete step:

- **Step 1 complete:** Duration selected
- **Step 2 complete:** Date AND time selected
- **Step 3 complete:** First name + email + Instagram + email acknowledgment signature matches full name + terms signature matches full name. If 25+ participants, high-traffic note required.
- **Step 4 complete:** Waiver signed
- **Step 5:** Always accessible once Steps 1-4 complete (add-ons are optional)

`getMaxAccessibleStep()` returns the highest step the user can navigate to.

Add-On Types and Rendering

Type	Rendering	State Shape
toggle	Single card with “Add / Added” toggle	<code>{ selected: boolean }</code>
quantity	+/- buttons + “All N” button	<code>{ quantity: number }</code>
tier	Chip row (mutually exclusive options)	<code>{ selection: string }</code>
backdrops	Horizontal carousel with “All” card + individual color cards	<code>{ mode: "none" "all" "single", colors: string[] }</code>
walls	Horizontal carousel with “All” card + individual wall cards	<code>{ mode: "none" "all" "single", walls: string[] }</code>

Event Booking Logic

- Events only available at **Powdersville**
- Only for **2+ hour** sessions (`supportsEvents: true` on duration)
- 1-hour sessions show a shake animation + toast if “Event booking” is selected
- 25+ participants triggers high-traffic note requirement
- 50+ participants triggers capacity alert (UI warning + owner email)
- Event acknowledgements required: cleanup, capacity understanding, self-service acknowledgment

Location Ordering

Powdersville is always first — in navigation, booking pages, gallery filters, and any UI presenting both locations. This is a standing product decision from Drew (2026-03-14).

9. Safety & Risk Analysis

Server-Side Pricing (Prevents Client-Side Manipulation)

The client sends add-on selections, but the **server looks up all prices** from `SESSION_PRICES` and `ADDON_PRICES` constants. A malicious client cannot change the payment amount.

HMAC Signing (Prevents State Tampering)

Booking state is HMAC-SHA256 signed before being embedded in the Square redirect URL. The callback verifies the signature using constant-time comparison (`crypto.timingSafeEqual`). A tampered state is rejected.

Appointment Type Allowlist

`VALID_APPOINTMENT_TYPE_IDS` is a hardcoded `Set` of known appointment type IDs. Any request with an unrecognized type ID is rejected with a 400 error.

Race Condition Window

There is a ~30 second window between payment confirmation and Acuity appointment creation. During this window, another customer could theoretically book the same slot.

- **At ~50 bookings/month**, the probability of collision is near zero
- If it occurs, `booking-callback.js` catches the Acuity error and redirects to `/booking-error?reason=slot-collision`
- Drew manually refunds via Square Dashboard

What Happens If...

Scenario	Result
Customer abandons Square checkout	Nothing happens. No appointment, no email, no charge.
Customer pays but slot is taken	Error page shown. Drew refunds manually via Square Dashboard.
Customer modifies state params in URL	HMAC verification fails → error page
Client sends fake price data	Ignored — server-side pricing is authoritative
Acuity API is down during callback	Appointment creation fails → error page, manual refund
Resend API is down	Owner notification silently fails (fire-and-forget), booking still succeeds
Square API is down during checkout	<code>create-checkout.js</code> returns 500, customer sees error alert

Why We Do NOT Auto-Cancel Unpaid Appointments

We don't create appointments before payment, so this is not an issue in the current architecture. In the earlier Block-then-Pay design, auto-cancel was considered but deferred because:

1. Acuity's `confirmationPagePaymentLink` has no expiry and cannot be invalidated
 2. Cancelling an appointment does NOT disable its payment link
 3. Vercel Hobby plan only supports daily crons (not 15-minute)
 4. At current volume, the risk is negligible
-

10. Costs

Service	Plan	Monthly Cost	Notes
Vercel	Hobby	\$0	100GB bandwidth, 100 hrs serverless, 6000 min build
Square	Pay-as-you-go	2.9% + \$0.30 per txn	On ~\$15K/month GMV: ~\$450/month
Acuity Scheduling	Business (via Squarespace)	Included in Squarespace subscription	API access included
Resend	Free tier	\$0	100 emails/day, 3000/month
QuickBooks Online	(Drew's existing subscription)	N/A	Already paying for this
GoDaddy	Domain registration	~\$20/year	whitewallstudios.co
PostHog	Free tier	\$0	Analytics (if added)

Vercel Hobby Plan Limits

- 100 GB bandwidth/month
 - 100 hours serverless function execution/month
 - 6,000 build minutes/month
 - Serverless functions: 10-second timeout
 - Cron jobs: daily only (no 15-minute intervals)
-

11. Environment & Deployment

How to Deploy

Push to main branch. Vercel auto-deploys.

```
git add .
git commit -m "description of changes"
git push origin main
```

Vercel creates a production deployment at `whitewallstudios.co` and a unique preview URL for each commit.

Preview Deployments

Every push to any branch creates a preview deployment at a unique URL (e.g., `white-wall-mockup-abc123.vercel.app`). Preview deployments use the same environment variables as production.

How to Test (Sandbox Mode)

Set `SQUARE_ENVIRONMENT=sandbox` in Vercel env vars. The system uses Square's sandbox API.

Test cards:

Card Number	CVV	Expiry	Result
4111 1111 1111 1111	111	Any future date	Success
4000 0000 0000 0002	Any	Any future date	Decline

Sandbox behavior note: The Square sandbox testing panel does not auto-redirect to the callback URL. You must manually click the redirect URL from the testing panel. In production, Square's checkout page redirects the browser automatically.

Going Live (Production Checklist)

1. Replace `SQUARE_ACCESS_TOKEN` with production token from Square Developer Dashboard
2. Replace `SQUARE_LOCATION_ID` with production location ID
3. Set `SQUARE_ENVIRONMENT=production`
4. Verify the `baseUrl` in `create-checkout.js` matches the production domain
5. Test one real booking end-to-end

Local Development

Static pages can be served with any HTTP server:

```
python3 -m http.server 8000
```

Note: Local server requires `.html` extensions in URLs. Vercel's `cleanUrls` handles clean routes in production.

Serverless functions (`api/`) only work on Vercel. For local API testing, use `vercel dev` (requires Vercel CLI).

12. Future Roadmap

QuickBooks Auto-Mark Invoices as Paid

- Complete Intuit compliance questionnaire (~30 min)
- Get production OAuth2 credentials
- Booking callback finds draft invoice by customer + date → records payment
- Tested successfully in sandbox (2026-03-20)

Acuity / Square Webhooks

- Real-time notifications when bookings are paid
- Detect payment on cancelled appointments (edge case safety net)
- Auto-update dashboard or CRM
- Acuity webhook events: `scheduled`, `rescheduled`, `canceled`, `changed`, `order.completed`
- Webhook payload: `application/x-www-form-urlencoded`, signed with HMAC-SHA256 using API key

Funnel Data Collection

- PostHog events for abandoned booking follow-up
- Track drop-off at each step of the booking flow

Admin Dashboard

- Live booking feed combining Acuity + Square data
- Revenue metrics, booking trends

A/B Testing

- PostHog feature flags + Claude optimization

Google Reviews

- Automated refresh cadence for review display on site
 - Research documented in `client/google-reviews-research.md`
-

13. Critical Warnings & Operational Rules

NEVER Do These Things

1. **NEVER bulk-modify production Acuity data** without per-item confirmation. One wrong API call can cancel real bookings or send incorrect emails to customers.
2. **NEVER cancel Acuity appointments via API** without understanding that the associated payment link CANNOT be invalidated. The customer can still pay on a cancelled appointment's payment page.
3. **NEVER change Acuity appointment types** without updating BOTH:
 - `scripts/booking-config.js` (client-side duration → type ID mapping)
 - `api/_lib/acuity.js` (`VALID_APPOINTMENT_TYPE_IDS`, `SESSION_PRICES`, `TYPE_TO_CALENDAR`, `TYPE_TO_DURATION`)
4. **NEVER change add-on prices in Acuity** without also updating `ADDON_PRICES` in `api/_lib/acuity.js`. The Square checkout will charge the old price.
5. **NEVER switch Square from sandbox to production** without testing the full flow end-to-end first. Verify:
 - `SQUARE_ACCESS_TOKEN` is the production token

- `SQUARE_LOCATION_ID` is the production location
- `SQUARE_ENVIRONMENT` is set to `production`
- `baseUrl` in `create-checkout.js` is correct

Known Broken Acuity Endpoints

- **POST `/appointments/{id}/payments`** — Returns 500 Internal Server Error for every `source` value. Tested 50+ values including `cash`, `square`, `check`, `external`, `manual`, `credit_card`, integers. This endpoint exists in Acuity’s router (returns 400 if `source` is missing) but is NOT in official API docs. Likely broken or internal-only.
- **`noEmail` parameter** on POST `/appointments` — Does NOT suppress Acuity’s confirmation email. Emails are still sent regardless.

Environment Variable Dependencies

If any of these are missing, the corresponding function will throw:

Missing Variable	Effect
<code>ACUITY_USER_ID</code> or <code>ACUITY_API_KEY</code>	All availability + appointment API calls fail
<code>SQUARE_ACCESS_TOKEN</code>	Checkout creation fails
<code>SQUARE_LOCATION_ID</code>	Checkout creation fails
<code>BOOKING_SECRET</code>	State signing/verification fails (checkout + callback)
<code>RESEND_API_KEY</code> or <code>NOTIFICATION_EMAIL</code>	Owner notifications silently skipped (non-fatal)
<code>QBO_CLIENT_ID</code> or <code>QBO_CLIENT_SECRET</code>	QuickBooks OAuth flow fails (not used in booking flow)

Appendix: File Reference

Static Pages

File	Route	Purpose
<code>index.html</code>	<code>/</code>	Homepage
<code>powdersville.html</code>	<code>/powdersville</code>	Powdersville venue page
<code>taylors-mill.html</code>	<code>/taylors-mill</code>	Taylor’s Mill venue page
<code>gallery.html</code>	<code>/gallery</code>	Photo gallery
<code>book-powdersville.html</code>	<code>/book-powdersville</code>	Powdersville booking flow
<code>book-taylors-mill.html</code>	<code>/book-taylors-mill</code>	Taylor’s Mill booking flow
<code>booking-confirmation.html</code>	<code>/booking-confirmation</code>	Post-booking success page
<code>booking-error.html</code>	<code>/booking-error</code>	Error/fallback page
<code>gear-rentals-powdersville.html</code>	<code>/gear-rentals-powdersville</code>	PV gear rentals
<code>gear-rentals-taylors-mill.html</code>	<code>/gear-rentals-taylors-mill</code>	TM gear rentals
<code>faq.html</code>	<code>/faq</code>	FAQ page
<code>props.html</code>	<code>/props</code>	Props page
<code>guides.html</code>	<code>/guides</code>	Guides page

File	Route	Purpose
privacy.html	/privacy	Privacy policy
terms.html	/terms	Terms of service
404.html	(Vercel default)	Custom 404 page

Serverless Functions

File	Route	Method	Purpose
api/availability-dates.js	/api/availability-dates	GET	Proxy Acuity available dates
api/availability-times.js	/api/availability-times	GET	Proxy Acuity available times
api/verify-availability.js	/api/verify-availability	POST	Pre-checkout slot verification
api/create-checkout.js	/api/create-checkout	POST	Build Square Payment Link
api/booking-callback.js	/api/booking-callback	GET	Post-payment: verify + create appointment
api/notify-owner.js	(internal module)	—	Send owner email for high-traffic bookings
api/qbo-auth.js	/api/qbo-auth	GET	QuickBooks OAuth2 authorization redirect
api/qbo-callback.js	/api/qbo-callback	GET	QuickBooks OAuth2 token exchange

Shared Libraries

File	Exports	Purpose
api/_lib/acuity.js	acuityGet, acuityPost, acuityDelete, isValidAppointmentTypeID, CALENDAR_IDS, TYPE_TO_CALENDAR, TYPE_TO_DURATION, ACUITY_ADDON_IDS, ACUITY_FIELD_IDS, buildAcuityAddonIDs, buildAcuityFields, buildAppointmentNotes, buildSquareLineItems, signState, verifyAndDecodeState	Acuity API wrapper, ID mappings, pricing, HMAC

File	Exports	Purpose
api/_lib/square.js	createPaymentLink, getOrder, deletePaymentLink, refundPayment	Square API wrapper